

Corona Tweet Classification Using NLP Models

Ido Rapaport , Eran Tasceme

August 2025

[GitHub](#)

1 Introduction

This work addresses sentiment analysis of Twitter posts about the COVID-19 pandemic. We downloaded two pre-trained models from the Hugging Face repository and adapted them to our task. The first, `cardiffnlp/twitter-roberta-base-sentiment-latest`, is a RoBERTa-base encoder pre-trained on large-scale Twitter data and fine-tuned for three-way sentiment; its chief advantages are strong contextual capacity and robustness to social-media artifacts (hashtags, mentions, URLs), making it well-suited to short, noisy tweets, while its drawbacks are a relatively heavy footprint and a default label space that may require remapping for alternative class schemes. The second, `distilbert/distilbert-base-uncased-finetuned-sst-2-english`, is a distilled, lighter BERT variant fine-tuned on SST-2; it offers low latency and a small memory footprint that facilitate deployment and ablation studies, but its binary label space and non-Twitter pretraining can limit out-of-the-box domain alignment. In our project, we performed transfer learning to fine-tune both models on our COVID-19 tweet dataset and align them with our target label space; Twitter-RoBERTa served as the accuracy-oriented, domain-specialized backbone, and DistilBERT as an efficient baseline and production-friendly alternative. To further improve efficiency, each fine-tuned model was subsequently compressed using a combination of quantization (to reduce numerical precision and memory), pruning (to remove redundant weights), and knowledge distillation (to transfer performance from a larger teacher to a smaller student), thereby reducing inference cost while preserving competitive predictive quality.

2 Exploratory Data Analysis

2.1 The Data

The Coronavirus Tweets NLP—Text Classification dataset released on Kaggle for supervised sentiment analysis. The dataset includes manual annotations of COVID-19-related tweets from the early pandemic period (approximately March–April 2020), capturing public discourse as the crisis unfolded. It is provided as two files, `Corona_NLP_train.csv` and `Corona_NLP_test.csv`, compiled from Twitter posts and labeled by human annotators. The training split contains 41,157 tweets and the test split 3,798 tweets, each with six fields (*UserName*, *ScreenName*, *Location*, *TweetAt*, *OriginalTweet*, *Sentiment*), enabling standardized experimentation and benchmarking across models. It contains five sentiment classes—*Extremely Negative*, *Negative*, *Neutral*, *Positive*, and *Extremely Positive*—making it suitable for multiclass sentiment classification benchmarks and model comparison. Unfortunately, the sentiment sample distribution is imbalanced, which may harm performance, as described in [1]. This article showed that when an LLM is fine-tuned on data where label frequencies aren't uniform across subgroups the model's latent space becomes more separated by subgroup, and the model learns to rely on uninformative features. They also demonstrate that per-group class weighted loss mitigates these harms—a method that was considered as an option to address this problem, but was abandoned due to the choice of another solution that will be detailed below. The train samples amounts are—*Positive* 11,422; *Negative* 9,917; *Neutral* 7,713; *Extremely Positive* 6,624; *Extremely Negative* 5,481—while the test split is comparatively Equivalent (1,041/947/619/599/592 across the same classes).

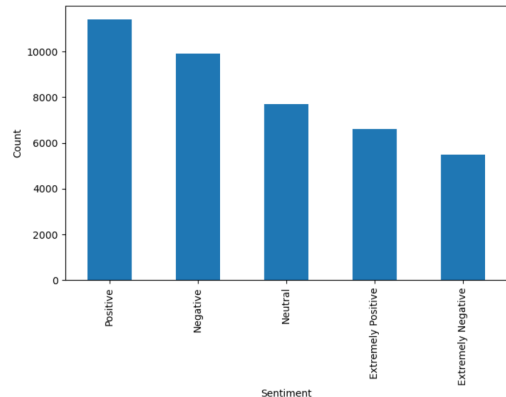


Figure 1: Sentiment distribution of the training data

2.2 Feature analysis

Nulls are concentrated entirely in *Location* (8,590 rows; 20.9%), whereas the remaining fields are effectively complete. In addition, we identified a lot of noise in this feature values, with many of them being location names that were spelled incorrectly or unevenly, so we chose to ignore this feature.

The features *UserName*, *ScreenName*, represent names of the tweet authors, and we found no contribution to these features when it comes to sentiment learning, so we chose to abandon them as well.

We found that the number of words in the tweets ranges from 1 to 64, which is a wide range. The length of the tweets also varies significantly, from 11 to 355 characters. Tweets raw length averages $\sim 30.5 \pm 11.6$ words (204 ± 68.7 characters); after standard normalization (lowercasing; removal of URLs, mentions, and punctuation) the mean length is ~ 18.6 words with a non-trivial tail of very short texts. However, the sentiment distribution appears stable regarding word count, with The neutral sentiment being an exception, and tends to have shorter tweets average than the other sentiments-but still within the normal tweets Length range.

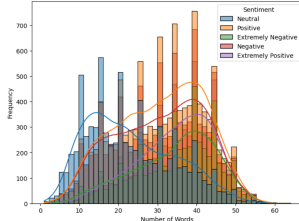


Figure 2: Words length distribution

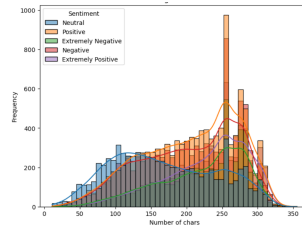


Figure 3: Chars length distribution

Post stop-word removal, lexical statistics are dominated by pandemic terms (e.g., *coronavirus*, *covid*) and consumer/retail vocabulary (*prices*, *food*, *supermarket*, *store*, *grocery*), alongside platform artifacts (e.g., *amp*).

Roughly 48.2% of tweets contain URLs, 23.9% include user mentions, and 57.6% include hashtags, and emojis are rare but exist. Word clouds and token counts by label indicate broad lexical overlap across sentiments, suggesting no single lexical feature is strongly diagnostic. Inspection of *TweetAt* (tweet dates) revealed no consistent association between date and label beyond expected event-driven spikes.

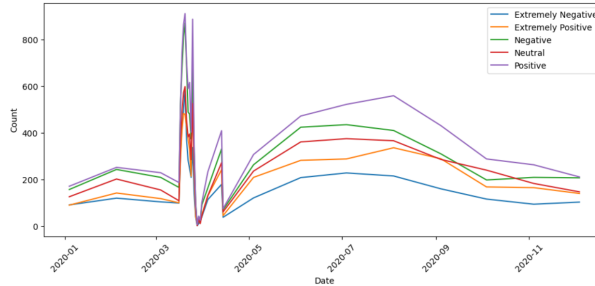


Figure 4: Sentiments counts over time

2.3 Data cleaning and normalization

For preparing the Twitter data for training a large language model, a series of normalization and data cleaning steps were performed to reduce noise and improve the quality of the dataset. For normalization and data cleaning, we lowercased all text to reduce sparsity and stabilize tokenization—an approach supported by studies showing uncased tokenization can improve performance when casing is inconsistent [2]. We also replaced user mentions and URLs with canonical placeholders (e.g., “@USER”, “HTTPURL”) to remove high-variance, low-semantic artifacts while preserving structural cues, exactly as done in BERTweet’s tweet-specific normalization pipeline [3]. For hashtags, we stripped only the “#”. We deleted HTML entities, collapsed repeated punctuation/whitespace, and extracted main text from pages to make tokenization deterministic. Finally, we de-duplicated duplicate lines to curb memorization and improve generalization, consistent with findings that dedup reduces verbatim copying by $10\times$ and yields better accuracy at the same compute [4]. Overall, these steps align with growing evidence that noisy training data (including mislabeled or structurally noisy text) measurably harms LLM performance and that

targeted filtering/selection helps—see controlled studies on noise degrading reasoning performance [5].

To quantify the effectiveness of the normalization/cleaning stage, Both models were evaluated on 3 different dataset variants, each representing a different level of pre-processing.

2.4 Augmentation

To address the issue of sentiment class imbalance, we explored several strategies. One option was to use per-group class-weighted loss, where each class is assigned a weight inversely proportional to its frequency, thereby penalizing misclassifications of minority classes more strongly and encouraging the model to treat them equally. A second option was to apply data augmentation on the train data, and since we observed in our earlier protocols that the models reached overfitting very quickly, even though we applied dropout regularization, we ultimately preferred the augmentation-based balancing approach over class-weighted loss. While adjusting the loss weights does not prevent memorization of the limited samples, data augmentation is a well established strategy for reducing overfitting, since it increases the effective diversity of the training set and forces the model to generalize better to unseen examples [6].

The approach we implemented was a combined augmentation pipeline: first performing back-translation (Hebrew $\rightarrow$ English $\rightarrow$ Hebrew) to generate semantically equivalent but lexically varied sentences, a method widely used to expand text corpora in low-resource settings [7], and then applying contextual word embedding substitution with a multilingual transformer (XLM-R) to replace individual words with contextually appropriate alternatives. By generating new minority-class samples through these two complementary augmentation techniques, we expanded the dataset until each sentiment label matched the frequency of the majority class, producing a balanced training set.

Training the models on the augmented data showed the best results and indeed reduced the degree of overfitting - so we used the augmented train for all models built afterwards.

3 Model Training Process

3.1 Training Setup

Two transformer-based language models, **Twitter-RoBERTa** and **DistilBERT**, were trained to perform sentiment classification on tweets. we trained the baseline models according to Exercise-4 form methodology under three matched protocols: (1) **No-processing**—trained on the raw corpus and evaluated on a matching unprocessed validation split; (2) **Cleaned**—trained and evaluated and tested on the processed corpus described above; and (3) **Cleaned + length-trim**—identical to (2) but with the samples on the extreme poles of text lengths removed from both train and validation (texts with <5 or >30 words; 5.56% of the split). In both model families, the processed settings outperformed the no-processing baseline, and the strongest results were obtained with **Cleaned + length-trim**, indicating a clear preference for training on cleaned data with the additional removal of extreme-length texts. After we received the results of the models trained on the various data processing protocols, we decided to use the third form of processing, the truncated one, and perform augmentation on its training data and examine the results the models that trained with this data.

3.2 Initial Results and Overfitting

The preliminary results revealed clear signs of overfitting. Training accuracy consistently exceeded 90%, while validation accuracy remained around 60%. Furthermore, the training curves exhibited smooth convergence across epochs, whereas the validation accuracy displayed significant variance, indicating unstable generalization.

Among the four dataset variants, the most stable results were obtained with the fourth dataset, which consisted of a cleaned, truncated, and augmented version of the original corpus. On the test set, Twitter-RoBERTa achieved 59% accuracy, while DistilBERT achieved 63%.

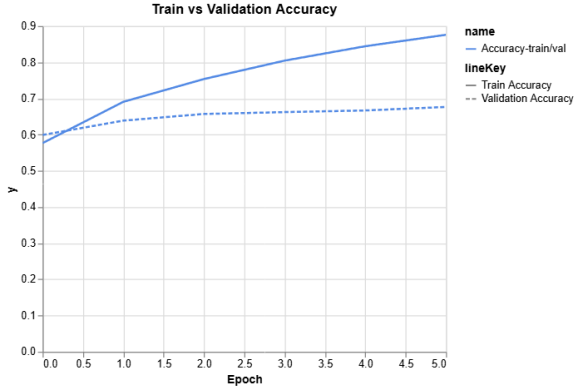


Figure 5: Training log from **Weights & Biases (WANDB)** shows the overfitting on the original dataset.

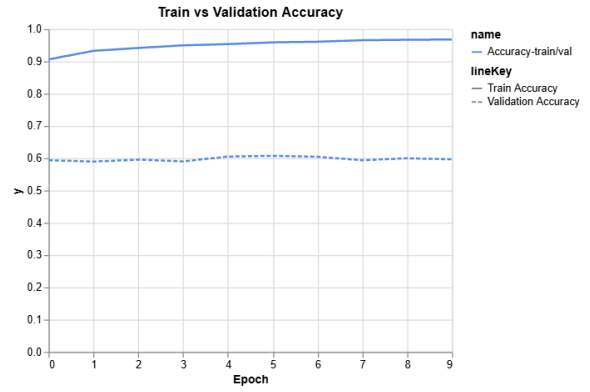


Figure 6: Training log from **Weights & Biases (WANDB)** for the cleaned, truncated, and augmented dataset shows a stable training

3.3 Refined Training Strategy

Given the constraints of time and computational resources, and based on the evidence that the cleaned and augmented dataset was the most effective, we refined our approach. We re-trained both models following the methodology of Exercise 5, this time exclusively on the fourth dataset.

This strategy leveraged the high-level training tools of the Hugging Face ecosystem, which provided more efficient optimization and stable performance compared to implementing the training loop solely in PyTorch. As a result, both models achieved approximately 74-75% accuracy on validation and test sets.

From each training methodology (Exercise 4 and Exercise 5), we selected the model that yielded the best results on the test set. In total, we retained four final models for comparison.

3.4 Hyperparameter Optimization with Hugging Face

Encouraged by these improvements, we conducted further experiments using Hugging Face’s hyperparameter optimization tools. The search included techniques such as early stopping, the use of different optimizers (AdamW and Adafactor), various learning rate schedulers (linear and cosine), mixed-precision training (fp16), and gradient accumulation.

For Twitter-RoBERTa, the best configuration used a learning rate of 4.06e-05, a batch size of 128, gradient accumulation of 1 step, weight decay of 0.143, the AdamW optimizer, and a linear learning rate scheduler. For DistilBERT, the optimal setup used a learning rate of 4.99e-05, a batch size of 128, gradient accumulation of 1 step, weight decay of 0.270, the AdamW optimizer, and again a linear learning rate scheduler.

Although validation accuracy reached as high as 90%, the test accuracy plateaued at approximately 75%. This indicates that while hyperparameter tuning improved the model’s ability to fit the validation data, its generalization to unseen data did not improve, reflecting a persistent challenge of overfitting.

3.5 Insights

The experiments demonstrated that while naive training procedures led to significant overfitting and limited generalization, leveraging Hugging Face’s optimized training pipelines and hyperparameter tuning markedly improved both stability and performance. As surveyed in the literature [8], a wide array of techniques exists to combat overfitting, of which we successfully implemented two for this project: data augmentation and the use of dropout layers. Our focus on these methods was further supported by the Exploratory Data Analysis (EDA), which confirmed a similar label distribution between the training and test sets, reinforcing our confidence in the representativeness of our data split. While we believe the models’ generalization and accuracy can be further improved, for the subsequent stages of this project, we will proceed with the four models that achieved an accuracy of 70-75%. The results show that training with Practice 5 achieved approximately 10% higher accuracy. Additionally, in both cases, the Distilbert model outperformed the roberta model by about 1%.

4 Model Compression

We applied several compression techniques to reduce the size and computational requirements of four selected language models while attempting to preserve performance.

4.1 Quantization and Pruning

Quantization and pruning were first applied, both achieving notable parameter reduction with only a small accuracy drop of about 1%, depending on the model. Quantization reduced the precision of weights from 32-bit floating point to 8-bit integers (using `dtype=torch.qint8`), lowering the size of DistilBERT from 66,957,317 to 23,854,080 parameters, and Twitter-RoBERTa from 124,649,477 to 39,037,440 parameters. For pruning, we removed 40% of the weights with the smallest magnitudes, a commonly used threshold that balances parameter reduction and model stability by preserving the most informative weights. After pruning, Twitter-RoBERTa was reduced to 90,437,371 non-zero parameters and DistilBERT to 49,732,915.

4.2 Knowledge Distillation

Knowledge distillation was also applied, transferring knowledge from a larger teacher model to a smaller student. Twitter-RoBERTa was distilled into DistilRoBERTa (82,122,245 parameters), and DistilBERT into TinyBERT (14,351,813 parameters), chosen for tokenizer compatibility to preserve input representations. We trained the students for three epochs with hyperparameters $\alpha = 0.5$ and temperature $T = 2.0$, a reasonable choice as these values balance the contribution of the teacher’s soft targets with the hard labels while smoothing the probability distribution. However, despite the theoretical benefits, both students achieved almost half the test accuracy of their teachers, showing poor generalization.

4.3 Low-Rank Factorization and Adaptation (SVD and LoRA)

Low-rank factorization via Singular Value Decomposition (SVD) was tested by decomposing weight matrices into smaller components, but dimensionality mismatches during implementation prevented us from completing evaluation. Although this method offers strong theoretical compression, its practical application proved complex within our scope. Finally, we experimented with Low-Rank Adaptation (LoRA), which inserts trainable low-rank matrices into existing layers. We trained for three epochs, but the results showed a significant degradation in accuracy, dropping by roughly half compared to the original models. While LoRA reduces the number of trainable parameters during fine-tuning, in our case it did not preserve predictive performance.

4.4 Summary

In summary, quantization and pruning provided the most stable trade-off between compression and accuracy, each reducing model size substantially with only minor performance loss. Distillation and LoRA, while conceptually powerful, resulted in significantly worse performance under our chosen settings. SVD presented technical challenges that could not be resolved in time. We acknowledge that further hyperparameter tuning and extended experimentation could likely improve results across all methods, but due to time and resource constraints, and given that some approaches already yielded satisfactory performance, we chose to proceed with the reported configurations.

5 Models Comparison and Selection

In evaluating the trained models, we aimed to consider not only accuracy on the test set but also factors related to efficiency and model complexity. To this end, we selected a set of comparison metrics following the recommendations of the study [9]. These metrics include accuracy, F1 score, Matthews Correlation Coefficient (MCC), Normalized Mutual Information (NMI), runtime all over the test set, total number of parameters, and non-zero parameters.

Since our goal was to maximize accuracy, MCC, and NMI, while at the same time minimizing runtime, total parameters, and non-zero parameters, we applied a normalization procedure to ensure comparability across different scales. For the metrics where lower values are preferable (runtime, total parameters, and non-zero parameters), we normalized them and then computed their reciprocal in order to align them

with the “higher is better” convention. Specifically, for each such metric x , the normalized value was computed as:

$$x_{\text{norm}} = \frac{1}{\left(\frac{x - \min(x)}{\max(x) - \min(x) + \epsilon} + 0.5 \right)} \quad (1)$$

where ϵ is a small constant added for numerical stability.

After normalization, the final score of each model was computed as a weighted aggregation of all metrics, using weights determined by their relative importance. Accuracy was assigned a weight of 0.4, MCC and NMI 0.2 each, runtime 0.1, and both total parameters and non-zero parameters 0.05. The final score is thus given by:

$$\text{Final Score} = 0.4 \cdot \text{Accuracy} + 0.2 \cdot \text{MCC} + 0.2 \cdot \text{NMI} + 0.1 \cdot \text{Runtime}_{\text{norm}} + 0.05 \cdot \text{TotalParams}_{\text{norm}} + 0.05 \cdot \text{NonZeroParams}_{\text{norm}} \quad (2)$$

This framework allowed us to holistically rank the models while balancing predictive performance, robustness, efficiency, and complexity. It also provided a clear basis for selecting candidate models for compression, ensuring that the models chosen for further optimization already demonstrated a favorable trade-off between accuracy and resource requirements.

Model	Accuracy	F1	MCC	NIT	Runtime (s)	Total Params	Final Score
(exc5) Quantized	0.749342	0.759337	0.681289	0.498732	81.179543	23854080	0.834644
(exc5) Pruned	0.747499	0.757465	0.678261	0.493489	111.000188	66957317	0.755882
(exc5)	0.750132	0.76129	0.682747	0.500303	112.623321	66957317	0.74863
(exc4) Quantized	0.604529	0.616722	0.497042	0.322598	73.134276	23854080	0.710575
(exc5) TinyBERT	0.406793	0.386043	0.316721	0.206525	15.833664	14351813	0.667367
(exc5) TinyBERT	0.373354	0.331221	0.288318	0.194163	16.137797	14351813	0.645317
(exc4) Pruned	0.61901	0.633854	0.518997	0.350883	122.14757	66957317	0.638605
(exc4)	0.630332	0.642991	0.533828	0.346945	116.28835	66957317	0.638409
(exc4) LoRA	0.28752	0.116906	0.044334	0.033471	130.14696	67699210	0.333542
(exc5) LoRA	0.280147	0.110438	0.028135	0.01953	135.851655	67699210	0.32213

Table 1: DistilBERT Results, The models were trained using the (exc) methodology

Model	Accuracy	F1	MCC	NIT	Runtime (s)	Total Params	Final Score
(exc5) Quantized	0.736177	0.74678	0.665713	0.479057	147.900836	39037440	0.755628
(exc5)	0.748289	0.75888	0.681719	0.498854	207.331913	124649477	0.67807
(exc4) Quantized	0.522643	0.54012	0.394144	0.248381	148.611607	39037440	0.569498
(exc5) DistilRoberta	0.537915	0.544353	0.439142	0.271259	113.883088	82122245	0.55594
(exc5) DistilRoberta	0.532122	0.53957	0.429216	0.263544	104.209126	82122245	0.555223
(exc4)	0.592417	0.6065	0.488801	0.306697	170.933801	124649477	0.548685
(exc5) Pruned	0.548183	0.464587	0.431959	0.38172	205.320812	124649477	0.533853
(exc4) LoRA	0.375197	0.330316	0.201075	0.132575	246.674712	125538826	0.350525
(exc4) Pruned	0.339389	0.273564	0.1888	0.098568	221.461156	124649477	0.341259
(exc5) LoRA	0.308057	0.194544	0.094354	0.085948	248.667155	125538826	0.292617

Table 2: Tweeter-Roberta Results, The models were trained using the (exc) methodology

6 Results And Conclusions

Our experiments revealed that the models based on DistilBERT consistently outperformed those based on Twitter-RoBERTa, despite having fewer parameters, and despite the fact that Twitter-RoBERTa was originally

pre-trained on semantic classification tasks. The best-performing model overall was DistilBERT, trained according to the protocol described in Exercise 5 and further compressed using quantization. A similar trend was observed for Twitter-RoBERTa, where the quantized version trained with the same procedure also achieved the strongest results among its variants. As illustrated in Figure 7, these quantized models dominate the performance space when evaluated in terms of both runtime efficiency and F1 score. Furthermore, analysis of the confusion matrix indicates that most misclassifications occur between semantically adjacent sentiment categories;

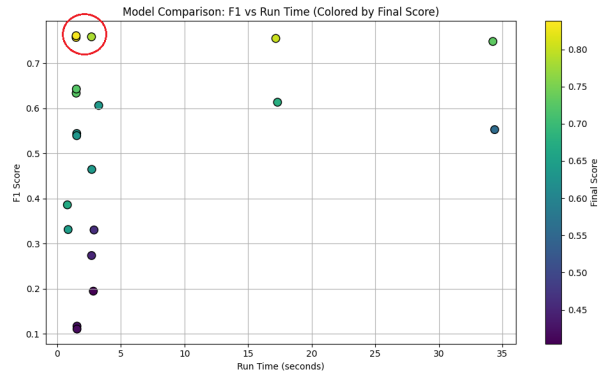


Figure 7: The red circle shows the Quantized Models with the highest scores also lead the pack in both F1 score and runtime.

for example, the model often predicts extremely positive or neutral instead of positive. We believe that even higher performance can be attained, and therefore recommend that future work expand the hyperparameter search space, conduct more extensive training, and explore a broader range of quantization parameters for model compression.

Actual	Extremely Negative	427	149	8	8	0
	Negative	85	752	52	141	11
	Neutral	6	58	494	61	0
	Positive	16	94	24	724	89
	Extremely Positive	1	23	0	133	442
		Extremely Negative	Negative	Neutral	Positive	Extremely Positive
		Predicted				

Figure 8: RoBERTa Quantized Confusion Matrix

7 References

- 1 Fyrillas, Andreas, et al. "Certified randomness in tight space." PRX Quantum 5.2 (2024): 020348.
- 2 Cory Paik and Katharina Von Der Wense. 2025. The Effectiveness of Uncased Tokenization for Clinical Notes. In Findings of the Association for Computational Linguistics: ACL 2025, pages 14986–14992, Vienna, Austria. Association for Computational Linguistics.
- 3 Nguyen, D. Q., Vu, T., Nguyen, A. T. (2020). BERTweet: A pre-trained language model for English Tweets. arXiv preprint arXiv:2005.10200.
- 4 Lee, K., Ippolito, D., Nystrom, A., Zhang, C., Eck, D., Callison-Burch, C., Carlini, N. (2021). Deduplicating training data makes language models better. arXiv preprint arXiv:2107.06499.
- 5 Havrilla, A., Iyer, M. (2024). Understanding the effect of noise in llm training data with algorithmic chains of thought. arXiv preprint arXiv:2402.04004.
- 6 Shorten, C., Khoshgftaar, T.M. A survey on Image Data Augmentation for Deep Learning. J Big Data 6, 60 (2019). <https://doi.org/10.1186/s40537-019-0197-0>
- 7 Sennrich, R., Haddow, B., Birch, A. (2015). Improving neural machine translation models with monolingual data. arXiv preprint arXiv:1511.06709.
- 8 Bejani, M.M., Ghatee, M. A systematic review on overfitting control in shallow and deep neural networks. Artif Intell Rev 54, 6391–6438 (2021). <https://doi.org/10.1007/s10462-021-09975-1>
- 9 Vickers, P., Barrault, L., Monti, E., Aletras, N. (2024). We need to talk about classification evaluation metrics in NLP. arXiv preprint arXiv:2401.03831.